

Regresión lineal e introducción al aprendizaje supervisado

Tomas Santana

September, 2026

Índice

1. Parte 1: Estructura de un problema de Aprendizaje Supervisado	1
1.1. Regresión vs Clasificación	1
1.2. Midiendo la calidad de un modelo	2
1.3. Definiendo un problema de aprendizaje supervisado	3
2. Parte 2: Regresión Lineal	4
2.1. Notación	4
2.2. El modelo lineal	5
2.3. Más de una variable de entrada	5
2.4. Una notación más conveniente	5
2.5. Función de costo	6
2.6. Algoritmo LMS	6
2.6.1. Derivada para un ejemplo	6
2.6.2. Más de un ejemplo de entrenamiento	7
2.7. Ecuaciones normales	9
2.7.1. Cálculo Matricial	9
2.7.1.1. Gradiente de un Escalar respecto a un Vector	9
2.7.1.2. Derivada de Vector respecto a Vector (Denominator Layout)	9
2.7.1.3. Regla de la Cadena para Funciones Escalares	10
2.7.1.4. Propiedades y Reglas Operativas Fundamentales	10
2.7.2. Solución analítica de mínimos cuadrados	10

1. Parte 1: Estructura de un problema de Aprendizaje Supervisado

En el dominio del aprendizaje supervisado, se busca aprender una función $\mathbf{x} \mapsto y$ a partir de un conjunto de entrenamiento $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$. Es decir, cada observación i del conjunto de entrenamiento está compuesta por un vector de características $\mathbf{x}_i$ y una etiqueta y_i . El objetivo es encontrar una función que pueda predecir la etiqueta y para nuevas observaciones basadas en sus características $\mathbf{x}$.

1.1. Regresión vs Clasificación

Toda variable puede ser de tipo cuantitativa o cualitativa (también llamada categórica).

Ejemplos de variables cuantitativas: precio de una casa, temperatura, altura de una persona, etc.

Ejemplos de variables cualitativas: el estado civil de una persona, el color de un auto, si una persona incumple el pago de un crédito, etc.

A los problemas supervisados asociados a predecir variables cuantitativas se les llama problemas de **regresión**, mientras que a los problemas supervisados asociados a predecir variables cualitativas se les llama problemas de **clasificación**.

Los métodos a utilizar dependen del tipo de variable que se desea predecir. Por ejemplo, para problemas de regresión se pueden utilizar la regresión lineal, mientras que para problemas de clasificación se pueden utilizar árboles de decisión, o regresión logística

Por otro lado, el tipo de las variables predictoras no tiene tanta influencia en la elección del método a utilizar.

1.2. Midiendo la calidad de un modelo

Para poder evaluar el desempeño de un modelo, se debe definir una métrica que mida la calidad de sus predicciones (cuantificando que tan cerca de la predicción está el valor real). Por ejemplo, para problemas de regresión la métrica más utilizada es el error cuadrático medio (MSE).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Cuando se computa el MSE sobre el conjunto de entrenamiento, se le llama **error de entrenamiento**, mientras que cuando se computa sobre un conjunto de prueba, se le llama **error de prueba**.

En general, el error de entrenamiento no nos interesa tanto como el error de prueba. Es decir, queremos saber **la precisión de las predicciones ante datos no vistos**.

- Para un modelo del mercado de valores, queremos saber que tan bien predice el precio de una acción en el futuro, no que tan bien predice el precio de una acción en el pasado.

De forma precisa, queremos el modelo que nos de el menor MSE de prueba, incluso si eso significa que el MSE de entrenamiento es mayor. Veamos un ejemplo en la siguiente figura:

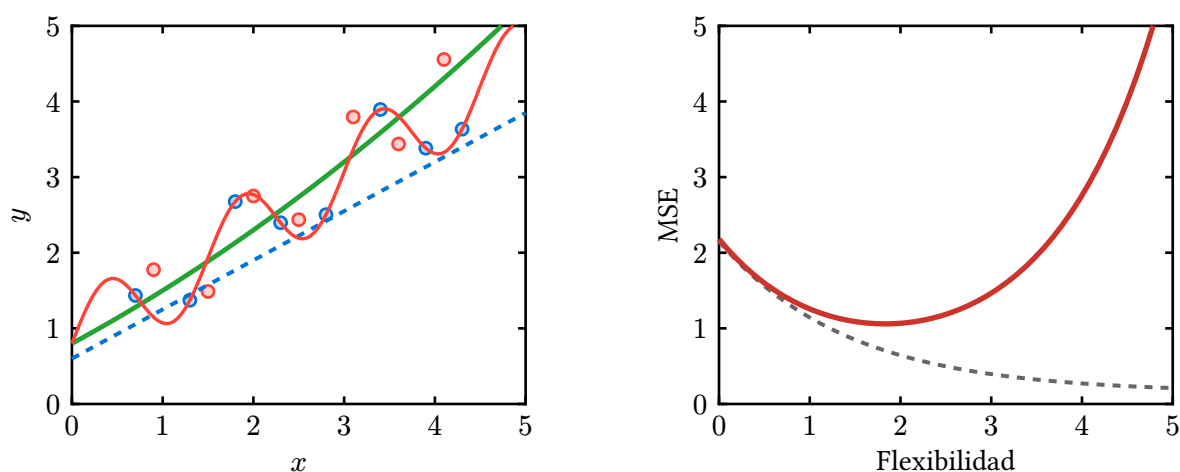


Figura 1: A la izquierda, se muestran varios ejemplos de funciones que podrían modelar los datos de entrenamiento. La función azul es un ejemplo de **underfitting**, la función verde es un ejemplo de **ajuste intermedio** y la función roja es un ejemplo de **overfitting**. A la derecha, se muestra el error cuadrático medio de entrenamiento (línea azul punteada) y prueba (línea roja) en función de la flexibilidad del modelo.

En la figura anterior, se puede observar que el modelo azul (underfitting) tiene un MSE de entrenamiento relativamente alto y un MSE de prueba también alto. El modelo verde (ajuste intermedio) tiene un MSE de entrenamiento más bajo y un MSE de prueba más bajo que el modelo azul. Finalmente, el modelo rojo (overfitting) tiene un MSE de entrenamiento muy bajo, pero su MSE de prueba es más alto que el del modelo verde.

Podemos ver que inicialmente, a medida que aumentamos la flexibilidad del modelo, el MSE de prueba disminuye, pero después de cierto punto, el MSE de prueba comienza a aumentar nuevamente.

Este ilustra **dilema sesgo-varianza** (bias-variance tradeoff), donde un modelo demasiado simple (alto sesgo) no captura la complejidad de los datos, mientras que un modelo demasiado complejo (alta varianza) se ajusta demasiado a los datos de entrenamiento y no generaliza bien a nuevos datos.

El sesgo es el error introducido por aproximar un problema real, que puede ser complejo, por un modelo más simple. La varianza corresponde a la sensibilidad del modelo a pequeñas fluctuaciones en el conjunto de entrenamiento. Más detalles sobre el dilema sesgo-varianza se puede encontrar en la [sección 2.2.2 de ISLP](#).

1.3. Definiendo un problema de aprendizaje supervisado

Suponiendo que tenemos un conjunto de datos sobre casas en Maracaibo, su área en metros cuadrados y su precio en dólares.

Área (pies²)	Precio (USD)
630	173875
966	229699
974	234262
1269	335333
1360	262383
1684	440726
2185	504177
2891	635097
...	...

Podemos graficar estos datos:

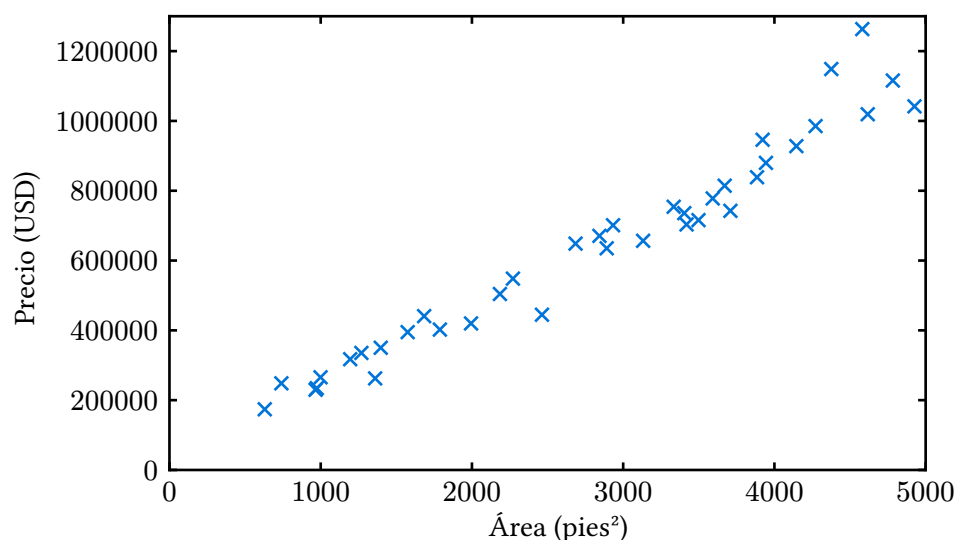


Figura 2: Área y precio de un conjunto de casas en Maracaibo.

Notación: $x^{(i)}$ denota un vector de características (o features), y $y^{(i)}$ denota la etiqueta (o label). El par $(x^{(i)}, y^{(i)})$ representa la observación i -ésima del conjunto de entrenamiento. Nótese que el superíndice (i) no tiene relación con la potencia, y es simplemente otra forma de indexar.

Para representar el espacio de valores de entrada se usa la letra $\mathcal{X}$, mientras que para representar el espacio de valores de salida se usa la letra $\mathcal{Y}$. En este caso, $\mathcal{X} = \mathcal{Y} = \mathbb{R}$.

Nuestro objetivo es encontrar una función $h : \mathcal{X} \rightarrow \mathcal{Y}$ tal que $h(x^{(i)}) \approx y^{(i)}$ para cada observación $(x^{(i)}, y^{(i)})$ del conjunto de entrenamiento. A esta función se le llama **hipótesis**.

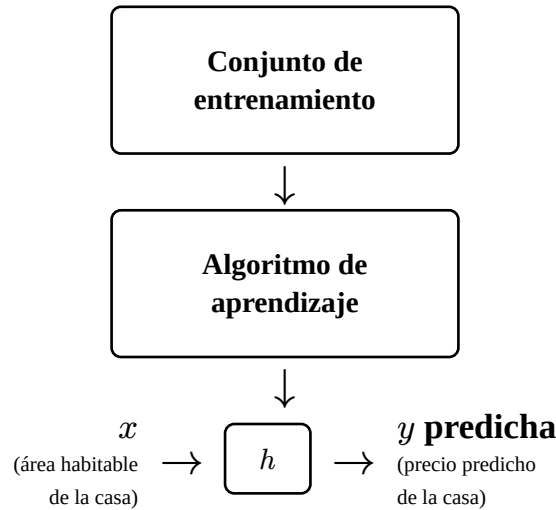


Figura 3: Esquema del proceso de aprendizaje supervisado. Extraído de las notas de [Stanford CS229](#).

El proceso de aprendizaje, como se observó, implica un objetivo de minimización de una función de pérdida que cuantifica el error entre las predicciones del modelo y los valores reales.

2. Parte 2: Regresión Lineal

La regresión lineal es uno de los modelos más sencillos de aprendizaje supervisado. Su objetivo es predecir una variable cuantitativa y a partir de una o más variables de entrada $\mathbf{x}$.

Continuemos con el ejemplo de las casas. Queremos utilizar información de una casa, como su área, para predecir su precio.

2.1. Notación

Supongamos que tenemos m ejemplos de entrenamiento:

$$S = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$$

donde:

- m es el número de ejemplos de entrenamiento.
- $\mathbf{x}^{(i)}$ representa las variables de entrada del ejemplo i .
- $y^{(i)}$ representa el valor que queremos predecir para el ejemplo i .
- θ representa los parámetros del modelo.
- n es el número de parámetros del modelo.

Por ejemplo, en nuestro problema de casas, si solamente utilizamos el área de la casa como entrada:

$$x_1^{(i)} = \text{área de la casa } i$$

y

$$y^{(i)} = \text{precio de la casa } i.$$

2.2. El modelo lineal

Ahora debemos decidir qué tipo de función utilizaremos para realizar las predicciones.

La regresión lineal supone que podemos aproximar y mediante una función lineal de las variables de entrada.

En el caso más sencillo, utilizando solamente el área de una casa:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1$$

2.3. Más de una variable de entrada

Podemos utilizar más información para realizar la predicción.

Por ejemplo, podríamos utilizar:

$$x_1 = \text{área de la casa}$$

$$x_2 = \text{número de habitaciones}$$

En ese caso, nuestro modelo sería:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

En general, un modelo lineal puede escribirse como:

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_{n-1} x_{n-1}$$

Aquí tenemos n parámetros:

$$\theta_0, \theta_1, \dots, \theta_{n-1}.$$

2.4. Una notación más conveniente

Para simplificar las ecuaciones, introducimos una variable adicional:

$$x_0 = 1$$

De esta manera podemos escribir:

$$h_{\theta}(\mathbf{x}) = \sum_{j=0}^{n-1} \theta_j x_j$$

Y si representamos los parámetros y las variables de entrada como vectores:

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n-1} \end{pmatrix}$$

y

$$\mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_{n-1} \end{pmatrix},$$

entonces podemos escribir todo el modelo simplemente como:

$$h_{\theta}(\mathbf{x}) = \theta^T \mathbf{x}.$$

2.5. Función de costo

Para encontrar los parámetros del modelo correctos, necesitamos definir una función que nos diga cuánto error comete el modelo en general. Una forma sencilla de medir el error consiste en elevar al cuadrado la diferencia entre cada predicción y su valor real:

$$(h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2.$$

Luego sumamos los errores de todos los ejemplos:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2.$$

A esta función $J(\theta)$ se le llama **función de costo**.

¿Por qué elevar al cuadrado la diferencia? Nos da dos beneficios: el primero es que errores positivos y negativos no se cancelan entre sí, y el segundo es que penaliza más los errores grandes que los pequeños.

Por otro lado, el factor $\frac{1}{2}$ no cambia qué valores de θ minimizan la función. Simplemente hará más convenientes algunas derivadas que veremos más adelante. Nuestro problema de aprendizaje puede entonces escribirse como:

$$\min_{\theta} J(\theta).$$

¿Cómo encontramos los valores de θ que minimizan $J(\theta)$?

2.6. Algoritmo LMS

Queremos escoger los parámetros θ de manera que minimicen la función de costo $J(\theta)$. Una forma de hacer esto es comenzar con algún valor inicial de θ e ir modificándolo poco a poco, buscando que $J(\theta)$ sea cada vez menor. Para esto utilizaremos el algoritmo de **descenso por gradiente**.

El descenso por gradiente actualiza cada parámetro θ_j utilizando:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

para cada $j = 0, \dots, n - 1$.

Aquí, α se conoce como la **tasa de aprendizaje**. La idea es que en cada paso modificamos los parámetros en la dirección que hace que $J(\theta)$ disminuya más rápidamente (es decir, en la dirección del gradiente negativo).

El símbolo $:=$ indica una actualización. Es decir, el nuevo valor de θ_j reemplaza al valor anterior.

2.6.1. Derivada para un ejemplo

Para poder aplicar el algoritmo necesitamos calcular:

$$\frac{\partial J(\theta)}{\partial \theta_j}$$

Comencemos con el caso más sencillo: supongamos que tenemos un único ejemplo de entrenamiento $(\mathbf{x}, y)$.

En ese caso, la función de costo es:

$$J(\theta) = \frac{1}{2} (h_{\theta(\mathbf{x})} - y)^2.$$

Queremos calcular su derivada con respecto a un parámetro θ_j :

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \left(\frac{1}{2} (h_{\theta(\mathbf{x})} - y)^2 \right)$$

Aplicando la regla de la cadena:

$$\frac{\partial J(\theta)}{\partial \theta_j} = (h_{\theta(\mathbf{x})} - y) \frac{\partial}{\partial \theta_j} (h_{\theta(\mathbf{x})} - y)$$

Como:

$$h_{\theta(\mathbf{x})} = \sum_{k=0}^{n-1} \theta_k x_k,$$

tenemos:

$$\left(\frac{\partial h_{\theta(\mathbf{x})}}{\partial \theta_j} \right) = x_j.$$

Por lo tanto:

$$\frac{\partial J(\theta)}{\partial \theta_j} = (h_{\theta(\mathbf{x})} - y) x_j.$$

Si sustituimos este resultado en la regla de descenso por gradiente:

$$\theta_j := \theta_j - \alpha (h_{\theta(\mathbf{x})} - y) x_j.$$

Esta expresión se conoce como la **regla de actualización LMS**. También puede escribirse como:

$$\theta_j := \theta_j + \alpha (y - h_{\theta(\mathbf{x})}) x_j.$$

Interpretación: si la predicción $h_{\theta(\mathbf{x})}$ está muy cerca del valor real y , el error será pequeño y los parámetros cambiarán poco. Por el contrario, si la predicción está muy lejos del valor real, el error será mayor y el cambio realizado sobre los parámetros también será mayor. Cada parámetro se modifica de acuerdo con:

$$\text{cambio} = \text{tasa de aprendizaje} \times \text{error} \times \text{entrada}.$$

2.6.2. Más de un ejemplo de entrenamiento

Hasta ahora derivamos la regla LMS suponiendo que teníamos un solo ejemplo. Sin embargo, normalmente tenemos un conjunto con m ejemplos.

Existen diferentes formas de utilizar estos ejemplos para actualizar los parámetros. Una primera opción consiste en utilizar todos los ejemplos de entrenamiento antes de realizar una actualización.

Recordemos que nuestra función de costo es:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta(\mathbf{x}^{(i)})} - y^{(i)})^2.$$

La derivada con respecto a θ_j es:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \sum_{i=1}^m (h_{\theta(\mathbf{x}^{(i)})} - y^{(i)}) x_j^{(i)}.$$

Por lo tanto, podemos utilizar el siguiente algoritmo de descenso por gradiente para actualizar los parámetros:

Repetir hasta convergencia:

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta(\mathbf{x}^{(i)})} - y^{(i)}) x_j^{(i)}$$

para cada $j = 0, \dots, n - 1$.

Agrupando cada actualización de los parámetros en un solo vector, podemos escribir:

$$\theta := \theta - \alpha \sum_{i=1}^m (h_{\theta(\mathbf{x}^{(i)})} - y^{(i)}) \mathbf{x}^{(i)}.$$

Este algoritmo se conoce como batch gradient descent. Nótese que el algoritmo de descenso de gradiente es susceptible a quedar atrapado en mínimos locales. Sin embargo, para el caso de regresión lineal, la función de costo es convexa, por lo que no hay mínimos locales y el algoritmo siempre encontrará el mínimo global (siempre que α no sea demasiado grande). La Figura 4 muestra las actualizaciones de los parámetros del algoritmo de descenso por gradiente sobre las curvas de nivel de la función de costo.

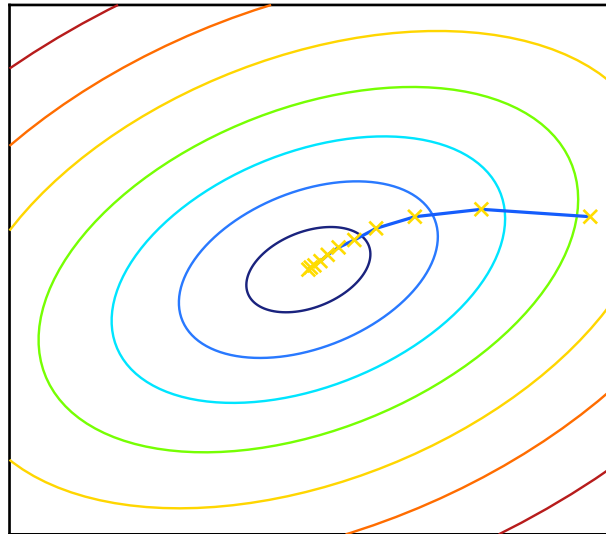


Figura 4: Curvas de nivel de la función de costo y pasos del algoritmo de descenso por gradiente hacia el mínimo.

Como alternativa al batch gradient descent, consideremos este otro algoritmo:

Repetir:

Por cada ejemplo de entrenamiento $(\mathbf{x}^{(i)}, y^{(i)})$ $1, \dots, m$:

$$\theta_j := \theta_j - \alpha (h_{\theta(\mathbf{x}^{(i)})} - y^{(i)}) x_j^{(i)}$$

para cada $j = 0, \dots, n - 1$.

Es decir, en lugar de esperar a que todos los ejemplos de entrenamiento sean evaluados antes de actualizar los parámetros, actualizamos los parámetros después de evaluar cada ejemplo. Este algoritmo se conoce como **stochastic gradient descent** (SGD). Este método es más rápido que el batch gradient descent, pero el camino hacia el mínimo es más ruidoso, ya que cada actualización depende de un solo ejemplo de entrenamiento.

Una tercera alternativa consiste en actualizar los parámetros después de evaluar un pequeño subconjunto de ejemplos de entrenamiento (por ejemplo, 32 o 64). Este algoritmo se conoce como **mini-batch gradient descent** (MBGD). Esta suele ser la opción más utilizada en la práctica, ya que combina las ventajas de BGD y SGD: no es demasiado costoso computacionalmente y su camino hacia el mínimo es más estable que el de SGD.

Para los casos de SGD y MBGD, el algoritmo nunca convergerá, pues nuevos ejemplos modificarán los parámetros ligeramente en una dirección específica. En este caso, se escoge un número de repeticiones específica, o se repite la actualización hasta que el costo no disminuya significativamente. Otra técnica es disminuir α poco a poco hasta llegar a 0.

2.7. Ecuaciones normales

En lugar de realizar actualizaciones usando descenso de gradiente, podemos obtener (en el caso de regresión lineal), el valor de los parámetros de forma explícita. Para derivar estas ecuaciones de forma simple, debemos introducir notación sobre como tomar derivadas de matrices y vectores.

2.7.1. Cálculo Matricial

Para una función $f : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}$, es decir, que mapea una matriz de dimensión $n \times d$ a los números reales, se define la derivada (o gradiente) de f respecto a una matriz $\mathbf{A}$ siguiendo el **denominator layout** (la derivada preserva las dimensiones de la variable):

$$\nabla_{\mathbf{A}} f(\mathbf{A}) = \begin{pmatrix} \frac{\partial f}{\partial \mathbf{A}_{11}} & \cdots & \frac{\partial f}{\partial \mathbf{A}_{1d}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial \mathbf{A}_{n1}} & \cdots & \frac{\partial f}{\partial \mathbf{A}_{nd}} \end{pmatrix}$$

Es decir, el gradiente $\nabla_{\mathbf{A}} f(\mathbf{A})$ es una matriz de tamaño $n \times d$, donde la entrada (i, j) es $\frac{\partial f}{\partial \mathbf{A}_{ij}}$.

Por ejemplo, si $\mathbf{A} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$ y $f(\mathbf{A}) = \frac{3}{2} \mathbf{A}_{11} + 5 \mathbf{A}_{12}^2 + \mathbf{A}_{21} \mathbf{A}_{22}$, el gradiente respecto a $\mathbf{A}$ es:

$$\nabla_{\mathbf{A}} f(\mathbf{A}) = \begin{pmatrix} \frac{3}{2} & 10 \mathbf{A}_{12} \\ A_{22} & A_{21} \end{pmatrix}$$

2.7.1.1. Gradiente de un Escalar respecto a un Vector

Como caso particular, para un vector columna $\mathbf{x} \in \mathbb{R}^{n \times 1}$ y una función escalar $f(\mathbf{x}) \in \mathbb{R}$, el gradiente adopta la forma del denominador (vector columna de $n \times 1$):

$$\frac{\partial f}{\partial \mathbf{x}} = \nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{n \times 1}$$

2.7.1.2. Derivada de Vector respecto a Vector (Denominator Layout)

Si tenemos una función vectorial $\mathbf{y} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, dada por $\mathbf{y}(\mathbf{x}) = [y_1(\mathbf{x}), \dots, y_m(\mathbf{x})]^T$, la derivada de $\mathbf{y}$ respecto a $\mathbf{x}$ en esta convención es una matriz de tamaño $n \times m$ (la transpuesta de la Jacobiana usual):

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \frac{\partial y_2}{\partial x_n} & \cdots & \frac{\partial y_m}{\partial x_n} \end{pmatrix} \in \mathbb{R}^{n \times m}$$

Donde la fila j contiene las derivadas respecto a x_j y la columna i corresponde a las variaciones de la componente y_i . Es decir:

$$\left[\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right]_{j,i} = \frac{\partial y_i}{\partial x_j}$$

Nota útil: Si $\mathbf{y} = \mathbf{x}$, entonces $\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = \mathbf{I}_n$ (la matriz identidad $n \times n$).

2.7.1.3. Regla de la Cadena para Funciones Escalares

Consideremos una función escalar compuesta $g(\mathbf{x}) = f(\mathbf{y}(\mathbf{x}))$, donde $\mathbf{y} \in \mathbb{R}^m$ y $f: \mathbb{R}^m \rightarrow \mathbb{R}$.

Aplicando la regla de la cadena en **denominator layout**:

$$\frac{\partial g}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial f}{\partial \mathbf{y}}$$

Verificación dimensional:

- $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{n \times m}$
- $\frac{\partial f}{\partial \mathbf{y}} \in \mathbb{R}^{m \times 1}$
- El producto resultante es de tamaño $(n \times m) \times (m \times 1) = n \times 1$, preservando la forma de $\mathbf{x}$.

2.7.1.4. Propiedades y Reglas Operativas Fundamentales

Sean $\mathbf{a} \in \mathbb{R}^m$ un vector constante, y $\mathbf{u}(\mathbf{x}), \mathbf{v}(\mathbf{x}) \in \mathbb{R}^m$ funciones vectoriales diferenciables:

1. **Forma lineal con vector constante ($\mathbf{a}^T \mathbf{y}$):** Como $\frac{\partial(\mathbf{a}^T \mathbf{y})}{\partial \mathbf{y}} = \mathbf{a}$, por la regla de la cadena:

$$\frac{\partial(\mathbf{a}^T \mathbf{y})}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{a}$$

2. **Regla del producto para producto punto ($\mathbf{u}^T \mathbf{v}$):** Cuando ambos vectores dependen de $\mathbf{x}$:

$$\frac{\partial(\mathbf{u}^T \mathbf{v})}{\partial \mathbf{x}} = \frac{\partial \mathbf{u}}{\partial \mathbf{x}} \mathbf{v} + \frac{\partial \mathbf{v}}{\partial \mathbf{x}} \mathbf{u}$$

En el caso particular de $\mathbf{x}^T \mathbf{y}$ (donde $\mathbf{u} = \mathbf{x}$ y por tanto $\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = \mathbf{I}$):

$$\frac{\partial(\mathbf{x}^T \mathbf{y})}{\partial \mathbf{x}} = \mathbf{I} \mathbf{y} + \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{x} = \mathbf{y} + \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{x}$$

3. **Norma al cuadrado / Producto consigo mismo ($\mathbf{y}^T \mathbf{y}$):** Dado que $\frac{\partial(\mathbf{y}^T \mathbf{y})}{\partial \mathbf{y}} = 2\mathbf{y}$, aplicando la regla de la cadena:

$$\frac{\partial(\mathbf{y}^T \mathbf{y})}{\partial \mathbf{x}} = 2 \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \mathbf{y}$$

2.7.2. Solución analítica de mínimos cuadrados

Para utilizar las herramientas del cálculo matricial para solucionar el problema de mínimos cuadrados, debemos definir el problema en términos de vectores y matrices. En primer lugar definimos la matriz de diseño $\mathbf{X}$ que contiene los ejemplos de entrenamiento como filas.

$$\mathbf{X} = \begin{pmatrix} -(\mathbf{x}^{(1)})^T & - \\ -(\mathbf{x}^{(2)})^T & - \\ \vdots & \\ -(\mathbf{x}^{(m)})^T & - \end{pmatrix}$$

Definamos $\mathbf{y}$ como un vector columna que contiene todas las etiquetas:

$$\mathbf{y} = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{pmatrix}$$

Sabemos que $h_{\theta}(\mathbf{x}^{(i)}) = (\mathbf{x}^{(i)})^T \theta$. Por lo tanto, podemos escribir el problema como:

$$\mathbf{X}\theta - \mathbf{y}$$

Usando el hecho que para un vector $\mathbf{u}$, $\mathbf{u}^T \mathbf{u} = \sum_i \mathbf{u}_i^2$, la función de costo:

$$\frac{1}{2}(\mathbf{X}\theta - \mathbf{y})^T (\mathbf{X}\theta - \mathbf{y}) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(\mathbf{x}^{(i)}) - y^{(i)})^2 = J(\theta)$$

Para minimizar esta función de costo, debemos derivar con respecto a θ y establecer la derivada igual a cero:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \frac{1}{2} (\mathbf{X}\theta - \mathbf{y})^T (\mathbf{X}\theta - \mathbf{y}) \\ &= \frac{1}{2} \nabla_{\theta} ((\mathbf{X}\theta)^T \mathbf{X}\theta - (\mathbf{X}\theta)^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\theta + \mathbf{y}^T \mathbf{y}) \\ &= \frac{1}{2} \nabla_{\theta} (\theta^T (\mathbf{X}^T \mathbf{X}) \theta - 2(\mathbf{X}^T \mathbf{y})^T \theta) \\ &= \frac{1}{2} (2\mathbf{X}^T \mathbf{X} \theta - 2\mathbf{X}^T \mathbf{y}) \\ &= \mathbf{X}^T \mathbf{X} \theta - \mathbf{X}^T \mathbf{y} \end{aligned}$$

En el paso 3, utilizamos el hecho que $\mathbf{a}^T \mathbf{b} = \mathbf{b}^T \mathbf{a}$. En el paso 5, utilizamos las siguientes reglas de derivación:

- $\nabla_{\mathbf{x}} \mathbf{b}^T \mathbf{x} = \mathbf{b}$
- $\nabla_{\mathbf{x}}^T \mathbf{A} \mathbf{x} = 2\mathbf{A} \mathbf{x}$ para una matriz simétrica $\mathbf{A}$.

Si hacemos la derivada igual a cero, obtenemos el punto crítico:

$$\mathbf{X}^T \mathbf{X} \theta = \mathbf{X}^T \mathbf{y}$$

Y si $\mathbf{X}^T \mathbf{X}$ es invertible, podemos despejar θ :

$$\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$